

Project one

Paula Solange Gund, Elia Sundström, Raji Nabih Youssef
(Dated: October 6, 2026)

Runge's function provides a useful test of polynomial regression, where increasing model complexity can improve the fit but also increase sensitivity to noise. We study ordinary least squares (OLS), Ridge and Lasso using 100 observations with additive noise. Bootstrap experiments with 100, 200, 500 and 1000 observations examine the bias-variance tradeoff. Five- and ten-fold cross-validation are used to compare polynomial degrees and regularization strengths. We also investigate gradient descent, automatic differentiation, momentum, AdaGrad, RMSProp, Adam and mini-batch updates. Degree-12 Ridge gives the lowest cross-validation MSE among the tested models: 0.0207671 with five folds and 0.0192712 with ten folds. The corresponding best OLS scores are 0.0211129 for five folds and 0.0202373 for ten folds, while Lasso gives 0.0211418 and 0.0201081. No optimization method requires the fewest updates in every case, and reaching the training-cost target does not guarantee the lowest prediction error. The results illustrate how polynomial degree, regularization and optimization settings affect prediction accuracy and convergence. The model rankings apply to the tested data and parameter, the cross-validation scores used for selection are not an independent final test.

CONTENTS

		B. Additional Results	17
		1. Part f	17
		2. Part g	17
		3. Part h	17
		4. Part i	17
		A. Introduction	
Abstract	1		
A. Introduction	1		
I. Methods	2		
A. OLS, Ridge and Lasso methods	2		
B. Cross-validation and bootstrap resampling	3		
C. The bias-variance tradeoff for OLS	4		
D. Plain gradient descent and automatic differentiation	4		
E. Momentum and adaptive gradient methods	5		
F. Lasso Regression and Subgradients	6		
G. Stochastic gradient descent	6		
H. Final model selection with cross-validation	7		
I. The Data Set	7		
J. Numerical methods	7		
II. Results and Discussion	7		
A. Ordinary Least Squared Regression	7		
1. Bootstrap resampling and the bias-variance tradeoff for OLS	8		
2. Cross-validation resampling for OLS	9		
B. Ridge regression	9		
1. Cross-validation for Ridge regression	10		
C. Plain gradient descent	11		
D. Momentum and adaptive gradient methods	12		
E. Lasso regression	13		
F. Stochastic gradient descent	14		
G. Final Model Selection	15		
III. Conclusion	16		
A. Use of AI/LLM Tools	16		
A.1 AI Tools used	16		
A.2 Text writing and editing	16		
A.3 Code generation and assistance	17		
References	17		

Making predictions from measured data requires a model that can capture the main patterns in the observations while still performing well on new data. Polynomial regression provides a simple and flexible way to approximate smooth functions, but its performance depends strongly on the polynomial degree. If the degree is too low, the model may be too simple to describe the underlying trend. If it is too high, the model can become overly sensitive to noise and produce unstable predictions. The goal is therefore not only to fit the observed data, but also to find a model that generalizes well.

Runge's function was used as the test case,

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]. \quad (1)$$

Although the function itself is smooth, high-degree polynomial interpolation at equally spaced points can produce large oscillations close to the boundaries. This phenomenon is referred to as Runge's phenomenon[1]. In this project, the function was instead sampled at random points and noise was added to the observations. This provided a useful setting for investigating how polynomial degree, regularization, and numerical conditioning influence prediction performance [2].

Three regression methods were studied: ordinary least squares (OLS), Ridge, and Lasso. OLS finds the model coefficients by minimizing the squared residuals. Ridge modifies this objective by adding a penalty on the squared coefficient values, while Lasso uses a penalty based on their absolute values. One important feature

of Lasso is that it can force some coefficients to become exactly zero. Ridge and Lasso were included because regularization can make the models less sensitive to noise and unstable coefficient estimates. At the same time, too much regularization can increase the bias by making the model less flexible and, consequently, less accurate.

The models were evaluated using a training/test data split, bootstrap resampling, and cross-validation. The training/test split provided an initial estimate of how well the models performed on unseen data. Bootstrap resampling was used to examine how predictions changed across resampled training sets and to study the bias-variance tradeoff. The squared-bias estimate was calculated using noisy test targets. Cross-validation was then used for model selection and provided a consistent way of comparing polynomial degrees and regularization strengths.

Numerical optimization of the regression models was also investigated. Closed-form solutions for OLS and Ridge were used as reference solutions, allowing the convergence of the iterative methods to be measured directly. Analytical gradients were derived and compared with gradients obtained through automatic differentiation to verify the implementation. The effect of the learning rate was also studied in relation to the curvature of the cost function.

Several gradient-based optimization methods were compared with standard gradient descent. Momentum makes use of information from previous updates, while AdaGrad and RMSProp adjust the effective step size based on accumulated squared gradients. Adam combines adaptive step sizes with momentum-like averaging. For OLS and Ridge, the methods were compared using the number of updates needed to reach a relative training-cost gap of 10^{-4} from the closed-form reference, together with their sensitivity to the initial learning rate.

Lasso required some extra care because its penalty is not differentiable when a coefficient is equal to zero[2]. A subgradient formulation was therefore used. The value produced by automatic differentiation at the non-differentiable point was also examined to determine whether it could be interpreted as a valid subgradient. This allowed the optimization study to be extended from OLS and Ridge to Lasso and made it possible to compare the behavior of all three regression methods under regularization.

Stochastic gradient descent was also studied to investigate the effect of using mini-batches rather than the full training set for each update. The batch size and learning-rate schedule were varied, and convergence was followed over 1,000 epochs to examine accuracy and computational cost. These results were compared with the full-batch methods, using the closed-form OLS and Ridge solutions as reference points.

Finally, OLS, Ridge, and Lasso were compared using the same five- and ten-fold cross-validation setup for all three methods. For OLS, the polynomial degree was selected. For Ridge and Lasso, both the polynomial de-

gree and the regularization strength were chosen. The best-performing model among those tested was identified from the cross-validated prediction error, and the differences between the methods were discussed in terms of model flexibility, coefficient shrinkage, and the bias-variance tradeoff.

The analysis was guided by three main questions: which regression model produced the lowest prediction error, which optimization method reached its reference solution most efficiently, and whether better optimization also resulted in better predictive performance.

All developed code referenced in this report is available via a GitHub repository ¹.

I. METHODS

Linear regression assumes that the model prediction can be written as

$$\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\vartheta}, \quad (2)$$

where the model is linear with respect to the parameter vector $\boldsymbol{\vartheta}$, $\tilde{\mathbf{y}}$ is the model prediction vector, and $\mathbf{X}$ the design matrix. Linear regression models are relatively simple and can, at times, outperform more advanced non-linear models, particularly when the data set is small, sparse, or the signal-to-noise ratio is low. This report will consider three linear regression models: ordinary least squared (OLS), Ridge, and Lasso [3].

Table I includes a list of the definition of used variables.

TABLE I: Global terminology and notation used in this report

Quantity	Sign
Design matrix	$\mathbf{X}$
Parameters	$\boldsymbol{\vartheta}$
Observations vector	$\mathbf{y}$
Predictions vector	$\tilde{\mathbf{y}}$
Noise vector	ε
Dataset size	n
Polynomial degree	p
Fold number in cross-validation	k
Ridge/Lasso penalty parameter	λ
Learning rate in gradient descent	η
Condition number	κ
Scikit-Learn's parameter	α

A. OLS, Ridge and Lasso methods

The general procedure in linear regression is to first define the cost function $C(\boldsymbol{\vartheta})$ and then to minimize it

¹ <https://github.com/elias98-ux/Machine-Learning---Project-1.git>

with respect to the parameter vector $\boldsymbol{\vartheta}$. This process is referred to as optimization, and this report will investigate various optimization methods. The cost function for OLS (3), Ridge (4), and Lasso (5) are

$$C_{\text{OLS}}(\boldsymbol{\vartheta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\vartheta}\|_2^2 \quad (3)$$

$$C_{\text{Ridge}}(\boldsymbol{\vartheta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\vartheta}\|_2^2 + \lambda \|\boldsymbol{\vartheta}\|_2^2 \quad (4)$$

$$C_{\text{Lasso}}(\boldsymbol{\vartheta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\vartheta}\|_2^2 + \lambda \|\boldsymbol{\vartheta}\|_1. \quad (5)$$

where λ is the regularization parameter. The OLS and Ridge regression has closed form solutions for parameters $\boldsymbol{\vartheta}$. Conversely, Lasso lacks a closed form solution due to the 1-norm not being differentiable at the origin [2].

OLS is unbiased, meaning that its estimator does not systematically over- or underestimate the true coefficients. In contrast, Ridge and Lasso regression are biased estimators, as the regularization term introduces a systematic deviation from the true coefficients. However, this bias is usually accompanied by a decrease in the variance, at least to a certain extent [2].

Consider the regularization for Ridge regression in terms of the singular-value decomposition (SVD) of the design matrix. Then each SVD mode is multiplied by a shrinkage factor

$$\frac{\sigma_j^2}{\sigma_j^2 + n\lambda}, \quad (6)$$

where the corresponding singular value is given by σ_i . Consequently, a mode with a large singular value is only weakly affected by the regularization. As opposed a modes with a small singular value, which is strongly suppressed. Therefore, increasing λ will reduce the contribution of the smaller modes. It is these modes that are more sensitive to a ill-conditioned design matrix. Ridge regression will thus suppress these ill-conditioned modes in at the cost of introducing a bias through the parameter λ . For a more detailed explanation, see the lecture book or the book *Elements of Statistical Learning* [2, 3].

The mean square error (MSE) is a measure of the difference between a true value and its estimate, it is given by

$$\text{MSE} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2, \quad (7)$$

where y_i is the targeted value and $\tilde{y}_i$ is the corresponding model prediction. Note that the MSE is the same as the cost function for OLS. The so-called coefficient of determination give an estimate of the quality of the fit, it is given by

$$R^2 = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2} \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i. \quad (8)$$

A perfect fit corresponds to $R^2 = 1$, while $R^2 = 0$ indicates the model performs no better than the mean of the target values. Negative values indicate the model performs worse than the prediction of the mean. The numerical value of MSE does not say anything on its own, as its magnitude depends on the scale of the target values. However, when combined with the R^2 it gives an measure of the model's predictive performance [2].

B. Cross-validation and bootstrap resampling

Once a model is fitted to a data set, resampling techniques can be used to gain some quantitative insight into the quality of the model. A method that can be used to check how much the model depends on the (somewhat arbitrarily) chosen test data set is the *bootstrap resampling* method. Here, we repeatedly draw a random subset of data points from the full data set. Each resampled data set is usually of the same number of data points, n , as the original dataset. This is done with replacement, meaning some data points can appear several times in one sample, while others are left out. The bootstrap resampling technique assumes independent, identically distributed observations and will not work for correlated data [2].

For k resamples of the dataset, the model is then applied to each sample, yielding k versions of the fitted parameters of the model. Each version varies a little bit depending on the data it was fitted on. These k parameters can now be assessed for variance giving an impression of how the model and data interact. In addition this gives a measure of the model sensitivity to variations in the data.

The bootstrap resampling technique then allows us to calculate variance and bias of the predictions of the model by comparing how much the predictions differ between the resamples. This is discussed in more detail in Section IC about the bias-variance tradeoff.

Another resampling technique with a slightly different focus is *cross-validation* resampling. The goal here is not to see how (in-)flexibly the model adapts to changes in the test data, but rather to gain insight into the quality of the model itself. It also allows for comparison between models with slightly different boundary conditions, such as polynomial degree of the OLS, or the size of the penalty parameter λ in Ridge regression (described in Section IA).

The *cross-validation* method splits the full dataset (test and training data) into k folds. Then, each fold serves as test data once while the model is fit on the $k - 1$ remaining folds. This is done k times until each fold has served as test data once. Note that for this to work, any scaling or centering of the data has to happen *after* the $k - 1$ folds are combined into one training dataset to ensure that the scaling/ centering is also adjusted to the data at hand. This scaling and centering is then reapplied to the test fold that is being held out and the MSE is calculated for each of the k folds. For equally

sized validation/ test folds V_j , the *cross-validation score* or *cross-validated MSE* can be written as

$$\text{CV}(p, \lambda) = \frac{1}{k} \sum_{j=1}^k \frac{1}{|V_j|} \sum_{i \in V_j} (y_i - \tilde{y}_i^{(-j)})^2, \quad (9)$$

where $\tilde{y}_i^{(-j)}$ is the prediction from a model fitted without using fold j . For OLS, the only model choice is the polynomial degree p , while Ridge and Lasso also depend on the regularization strength λ .

In our application of the bootstrap resampling technique, we applied it to the $n \in \{100, 200, 500, 1000\}$ data points of the training data and $k = 100$ resamples were performed. The resulting parameters are compared against the test data. For the resampling, we used the function `resample` from `sklearn.utils`[4]. In our application of cross-validation resampling, an additional comparison was made between a manual k -fold loop implementation and the pre-defined function `cross_val_score` from `sklearn.model_selection`. The number of folds was varied between $k = 5$ and $k = 10$.

For both the bootstrap and the cross-validation resampling, the MSE is calculated as a function of model complexity (polynomial degree p). The results are compared in Section II B 1.

C. The bias-variance tradeoff for OLS

The cost function that is to be optimized in OLS is given by

$$C(\mathbf{X}, \boldsymbol{\theta})_{OLS} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] \quad (10)$$

This expression is worth some further investigation. Inserting the expression for $y = f(x) + \varepsilon$ into the RHS of (10), we can expand it to become

$$\begin{aligned} \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \mathbb{E}[(f + \varepsilon - \tilde{\mathbf{y}} + \mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\tilde{\mathbf{y}}])^2] \\ &= \mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}]) - (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}]) + \varepsilon]^2 \\ &= \mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])^2 + (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2 + \varepsilon^2 \\ &\quad + 2\varepsilon(f - \mathbb{E}[\tilde{\mathbf{y}}]) - 2\varepsilon(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}]) \\ &\quad - (f - \mathbb{E}[\tilde{\mathbf{y}}])(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])], \end{aligned}$$

where we have added and subtracted $\mathbb{E}[\tilde{\mathbf{y}}]$ in the first step. Using the expectation value's additive properties, all the terms in the argument of the expectation value can be treated separately. The two cross terms then vanish due to the fact that, per definition in our model, $\mathbb{E}[\varepsilon] = 0$. For a non-stochastic f , we also have

$$\begin{aligned} \mathbb{E}[\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}]] &= \mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\mathbb{E}[\tilde{\mathbf{y}}]] \\ &= \mathbb{E}[\tilde{\mathbf{y}}] - \mathbb{E}[\tilde{\mathbf{y}}] = 0, \end{aligned}$$

leaving the remaining terms

$$\begin{aligned} \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])^2 + (\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2 + \varepsilon^2] \\ &= \underbrace{\mathbb{E}[(f - \mathbb{E}[\tilde{\mathbf{y}}])^2]}_{\text{Bias}^2[\tilde{\mathbf{y}}]} + \underbrace{\mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2]}_{\text{var}[\tilde{\mathbf{y}}]} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\sigma^2}, \end{aligned}$$

leaving us with the standard form

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \text{Bias}^2[\tilde{\mathbf{y}}] + \text{var}[\tilde{\mathbf{y}}] + \sigma^2, \quad (11)$$

which is called the *bias-variance decomposition*[2].

This decomposition contains the bias, which is a measure of how far the average prediction $\mathbb{E}[\tilde{\mathbf{y}}]$ is from the true value f that we are looking for. The bias measures the systematic error which is introduced by approximating the underlying function with a given model. Further, Eq. (11) contains the variance of the predictions $\tilde{\mathbf{y}}$, which is a measure of how scattered the predictions are between resampling, meaning that it describes how stably the model can predict values between samples. Lastly, there is the so-called irreducible error σ^2 . This error is the variance of the noise of the data itself and it is called irreducible for a reason: it serves as the lower boundary for the precision that any sensible model will be able to achieve, as no model could or should be able to get beyond the noise of the data itself.

The bias and the variance behave somewhat inversely to each other: a more complex model will be able to fit data more precisely, leading to lower bias, which in return will enhance the variance between resamples due to the fact that a more complex model will adapt more flexibly to even small, stochastic changes in the data. In the opposite case of a very simple, say linear model, resampling will often return very similar parameters even when the data varies, meaning we obtain a small variance, but can be off entirely depending on what the underlying data is, leaving us with large bias. The full error of our model containing the sum of their parts, the smallest error is usually obtained somewhere in between. This push and pull between variance and bias is what gives the phenomenon the name *bias-variance tradeoff*.

D. Plain gradient descent and automatic differentiation

Gradient decent focuses on finding a minimum of a general function $F(x)$ and works on the principle that a generally well-behaved function $F(x)$ decreases locally fastest when moving away from x along the negative gradient. In our case, we want to travel along the cost function $C(\boldsymbol{\theta})$ to obtain the parameters $\boldsymbol{\theta}$ that minimize it. For a so-called learning rate $\eta > 0$ that is small enough, this can be applied iteratively with an updating scheme

$$\vartheta_{k+1} = \vartheta_k - \eta \nabla_{\vartheta} C(\vartheta_k) \quad (12)$$

This method works well when the cost function is convex. If this is not the case, one is at risk of getting stuck at

a local minimum or saddle point, and other methods are needed [2].

Further, the learning rate η cannot be chosen arbitrarily. It can be derived that convergence of gradient descent is only given when the learning rate is within the region

$$0 < \eta < \frac{2}{\lambda_{\max}} \equiv \eta_{\max} \quad (13)$$

where $\lambda_{\max}$ is the largest eigenvalue of the Hessian matrix H of the cost function.

Additionally, the convergence can happen at different speeds. To quantify this, one can look at the *condition number* κ

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (14)$$

where, analogously to $\lambda_{\max}$, $\lambda_{\min}$ is the smallest eigenvalue of the Hessian matrix H . This fraction thus tells us how large the difference is between the steepness of the different directions, with large values of κ indicating a large difference between $\lambda_{\min}$ and $\lambda_{\max}$, and thus a large difference in steepness in the directions, and vice versa for small values of κ . The condition number κ can therefore give us a hint of how fast the gradient descent can be expected to converge, with large values indicating poor conditioning of the Hessian matrix H , and thus slow convergence.

While the calculation of the gradient $\nabla_{\boldsymbol{\vartheta}} C(\boldsymbol{\vartheta})$ is straightforward in theory, the analytical calculation of this gradient is not always feasible (or reasonable) in numerical implementations, especially when the cost function C becomes more complex.

It is therefore instructive to use *automatic differentiation*, which allows for a more efficient calculation of derivatives. The basic concept behind automatic differentiation is that any function is split into its underlying elementary operations (addition, subtraction, multiplication) and functions (exp, log) with known derivatives. The chain rule is then applied repeatedly, turning each partial differentiation into a simple derivative of a fundamental operation or function. Automatic differentiation thus is neither symbolic or numerical, as it never builds a full symbolic function of the derivative, nor does it use some numerical approximation scheme (such as e.g. finite differences) to approximate the derivative.

In our implementation of the plain gradient descent, the analytical and automatic differentiation were compared and agreed to machine precision. Therefore, all results produced for plain gradient descent presented in this report used automatic differentiation unless stated otherwise.

Further, a maximum number of iterations $N = 10^4$ was used to truncate the updating scheme to catch when convergence was too slow. If a difference between the closed form parameters $\boldsymbol{\vartheta}$ calculated as described in Section I A and the parameters $\boldsymbol{\vartheta}$ calculated by gradient descent reached a tolerance of $t = 10^{-8}$ before reaching this

iteration number, the gradient descent was broken off at this tolerance.

The gradient descent was then calculated for 6 different values of the learning rate η for both OLS and Ridge regression, with 5 values evenly spaced between 0.1 and $0.99\eta_{\max}$, and a last one at $1.01\eta_{\max}$.

E. Momentum and adaptive gradient methods

Momentum keeps part of the previous update and carries it into the next step. AdaGrad and RMSProp instead adjust the size of each coefficient update using information from the squared gradients seen so far. Let $g_t = \nabla C(\boldsymbol{\vartheta}_t)$.

All vector products and divisions below are applied element by element[2]:

$$v_{t+1} = \beta v_t + \eta g_t, \quad \vartheta_{t+1} = \vartheta_t - v_{t+1}, \quad (15)$$

$$\begin{aligned} r_{t+1} &= r_t + g_t^2 \quad (\text{AdaGrad}), \\ r_{t+1} &= \rho r_t + (1 - \rho) g_t^2 \quad (\text{RMSProp}), \end{aligned} \quad (16)$$

and

$$\vartheta_{t+1} = \vartheta_t - \eta \frac{g_t}{\sqrt{r_{t+1}} + \epsilon}. \quad (17)$$

AdaGrad keeps track of all earlier squared gradients, which means that its steps can gradually become very small. RMSProp avoids this by giving less weight to gradients from the distant past. Adam combines RMSProp and momentum by keeping moving averages of the gradients and of the squared gradients, while also correcting for the bias introduced at the beginning of the optimization. For Adam, the iterations are indexed from $t = 1$, with g_t evaluated at ϑ_{t-1} . The update rules are

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & r_t &= \beta_2 r_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{r}_t &= \frac{r_t}{1 - \beta_2^t}. \end{aligned} \quad (18)$$

$$\vartheta_t = \vartheta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{r}_t} + \epsilon}. \quad (19)$$

To compare convergence, we measured the relative training-cost gap,

$$q_t = \frac{\max(0, C(\vartheta_t) - C(\vartheta_{\text{ref}}))}{C(\vartheta_0) - C(\vartheta_{\text{ref}})}. \quad (20)$$

Here, ϑ_0 is the initial coefficient vector and ϑ_{ref} is the closed-form OLS or Ridge solution. The gap measures the remaining difference from the reference cost relative to the initial difference. The maximum with zero prevents small negative values caused by numerical rounding. We recorded the first update at which $q_t \leq 10^{-4}$, with a maximum of 20 000 updates per run.

These methods can change how quickly the optimization converges without changing the object being minimized. Their cost values do not necessarily decrease at every update, and the stability limit that applies to plain gradient descent is not a general bound for all adaptive methods [2].

Setting the momentum parameter to zero gives back plain gradient descent. Momentum can help maintain progress in directions where the gradients remain consistent, while the adaptive methods rescale the individual coefficient updates using the history of squared gradients. For the convex regression problems considered here, the main purpose of these methods is to improve numerical convergence rather than to escape non-global local minima. The small positive constant ϵ is included to avoid division by zero.

Plain gradient descent, momentum, AdaGrad, RMSProp, and Adam were compared for polynomial degrees 5, 8, and 15 using both OLS and Ridge regression. For Ridge, the mean-cost penalty was set to 0.1. Each method was initialized at zero and tested with nine logarithmically spaced learning rates between 10^{-4} and 1, with a maximum of 20 000 updates per run. The momentum parameter was set to $\beta = 0.9$, RMSProp used $\rho = 0.99$, and Adam used $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The small denominator offset was set to $\epsilon = 10^{-8}$ for Adam.

F. Lasso Regression and Subgradients

For Lasso regression, the OLS cost is extended with a 1-norm penalty on the model coefficients[2],

$$\begin{aligned} C_L &= C_{\text{OLS}} + \lambda \|\boldsymbol{\vartheta}\|_1, \\ g_L &= \frac{2}{n} \mathbf{X}^T (\mathbf{X} \boldsymbol{\vartheta} - \mathbf{y}) + \lambda \mathbf{s}, \\ s_j &\in \partial |\vartheta_j|. \end{aligned} \quad (21)$$

For a coefficient that is non-zero, the corresponding subgradient component s_j is simply given by its sign. At zero, however, the absolute-value function is not differentiable in the usual sense, and any value in the interval $[-1, 1]$ is a valid subgradient. This is what allows the 1-norm penalty to produce coefficients that are exactly zero at an optimum. In practice, however, ordinary subgradient updates with a finite step size do not necessarily land exactly on zero. With a constant step size, an update can instead move back and forth across the non-differentiable point [2].

Setting some coefficients to zero can make the fitted model simpler and easier to interpret, especially when some features contain little useful information. At the same time, polynomial features are often strongly correlated, and this can make the particular terms selected by Lasso sensitive to the data. The familiar soft-thresholding result applies directly when the design is orthogonal, or within a coordinate-wise update. For a general correlated design, it is therefore not correct to

obtain the Lasso solution by simply applying a threshold to each OLS coefficient.

The same five update methods and the nine learning rates were used for Lasso, this time with the analytical Lasso subgradient. Polynomial degrees 5, 8, and 15 were tested together with mean-cost penalties of 0.001, 0.01, and 0.1. The update budget and cost target were kept the same as for the previous gradient methods, giving a total of 405 Lasso runs.

A numerical reference solution was obtained using `sklearn.linear_model.Lasso` on the scaled features and centered targets, with `fit_intercept=False`, `max_iter=200000`, and `tol=1e-12` [4]. The parameter was set to `alpha =  $\lambda/2$` because Scikit-Learn uses a squared-error term divided by $2n$, whereas our cost uses division by n . This makes the two objectives equivalent up to a constant factor. This differs from what we did in part I E, which reports the fewest updates needed to reach the target.

G. Stochastic gradient descent

Mini-batch gradient descent replaces the gradient computed from the full data set with a gradient based on a smaller batch B . For Ridge or Lasso, the penalty gradient is then added once and is not divided again by the batch size[2].

$$g_B^{\text{OLS}} = \frac{2}{|B|} \mathbf{X}_B^T (\mathbf{X}_B \boldsymbol{\vartheta} - \mathbf{y}_B), \quad \eta_t = \frac{t_0}{t + t_1}. \quad (22)$$

One epoch corresponds to processing the training observations once. Using small batches reduces the amount of work needed for each update, but it also means that more updates are carried out during an epoch. Because each mini-batch contains only part of the data, the resulting gradient changes depending on which observations are sampled. A decreasing learning rate can help reduce these fluctuations later in the optimization, although it can also make further progress slower. Full-batch gradient descent does not have this mini-batch sampling variation. It is also important to note that using the same number of epochs does not mean that two methods have performed the same number of parameter updates or required the same runtime[2].

The main advantage of mini-batches is that each gradient evaluation uses fewer observations. This can make it possible to update the parameters more frequently, but the variation introduced by sampling can also make accurate convergence more difficult. For the convex problems considered here, the usefulness of stochastic gradient descent cannot be explained by an ability to escape local minima. Instead, any advantage has to be supported by the measured accuracy and computational cost. For the stochastic gradient descent experiments, the polynomial degree was fixed at 8. OLS, Ridge with penalty 0.1, and Lasso with penalty 0.001 were all tested using the same

five update methods. Three batch sizes were considered: 5, 20, and 80, where a batch size of 80 corresponds to full-batch descent. Each run started from zero coefficients and used 1 000 epochs with shuffle seed 2026. The base learning rate was set to 0.01. Two learning-rate schedules were compared: a constant rate and

$$\eta_t = \frac{1}{t + 100},$$

where t counts parameter updates starting from one. Altogether, this resulted in 90 runs. Within each epoch, the training data were shuffled without replacement, while the row order for the full-batch case remained fixed.

The coefficient values were saved at the end of each epoch and were later used to compute the relative training-cost gaps. Runs continued through the full epoch budget unless a numerical stability check stopped them earlier. The recorded runtime covered the training loop and the storage of the coefficient history, calculation of the reference solutions, or the later evaluation of the saved cost histories. With 1 000 epochs, the completed runs used 16 000 updates for batch size 25, 4 000 updates for batch size 100, and 1 000 updates for batch size 80. The effect of the number of epochs was therefore studied from the saved optimization trajectories rather than by carrying out a separate search over different epoch budgets.

H. Final model selection with cross-validation

The final model selection is based on the tested setting that gives the lowest cross-validation score:

$$\hat{p}_{\text{OLS}} = \arg \min_p \text{CV}_{\text{OLS}}(p), \quad (\hat{p}, \hat{\lambda}) = \arg \min_{p, \lambda} \text{CV}(p, \lambda). \quad (23)$$

Increasing the polynomial degree can reduce approximation bias because the model becomes more flexible, but this can also increase the variance of the fitted model. Ridge and Lasso address this in a different way by restricting the coefficient values, which can make the fitted model less sensitive to changes in the training sample.

Cross-validation measures the overall prediction error, so it does not separate the error directly into bias and variance contributions. Those components have to be interpreted together with the bias-variance analysis carried out earlier. It is also worth keeping in mind that when the same cross-validation scores are used both to select a model and to report its performance, the minimum selected score can be somewhat optimistic.

For the final model selection, all 100 observations were used five folds use 80 training/20 validation observations per fold and for ten folds use 90/10. Polynomial degrees from 0 to 19 were tested. OLS was evaluated at every degree. Ridge and Lasso were evaluated over both degree and penalty.

For degree zero, the model simply predicted the mean of the training targets. For the higher degrees, the polynomial processing with scaling and centering based only on the training fold was used.

For every tested setting, the mean validation MSE was calculated using the same folds for all three methods. The lowest cross-validation error was found separately for OLS, Ridge, and Lasso, and these three minimum values were then compared. In the degree comparison plot, Ridge and Lasso were shown using the best tested penalty for each degree. In the penalty plots, the polynomial degree was held fixed at the selected degree for that method. The selected models were not refitted and tested on a separate final holdout set.

I. The Data Set

The data applied in the code was generated by Runge's function (Eq. (1)) combined with a noise term that follows a gaussian distribution with the mean value 0 and the standard deviation $\sigma = 0.1$. The used data set has $n = 100$ different points.

To get an unbiased evaluation of a given model, it is necessary to split the data into training and test sets, with the evaluation of the model being performed on the unseen test data. In this report, the data is divided into an 80/20 training/test split. The data is also scaled, which is especially important for models with regularization, as the penalty depends on the magnitude of the model coefficients. Additionally, the data is centered to avoid penalizing the intercept. The code for this report use the function `StandardScaler` from the Scikit-learn library [4], which use

$$x_{\text{scaled}} = \frac{x - \bar{x}}{\sigma}, \quad (24)$$

where $\bar{x}$ is the mean of the data and σ its standard deviation [2].

J. Numerical methods

Throughout this project, the Scikit-Learn library [4] has been used to evaluating the regression models, while the JAX python library [5] has been used for automatic differentiation. For plotting the library Matplotlib [6] was used.

II. RESULTS AND DISCUSSION

A. Ordinary Least Squared Regression

The best performance for the OLS regression for the Runge function is given by a degree 8 polynomial, with a test MSE of 0.0187, see the left plot in FIG. 1 a). The corresponding test R^2 score for the optimal degree is 0.849.

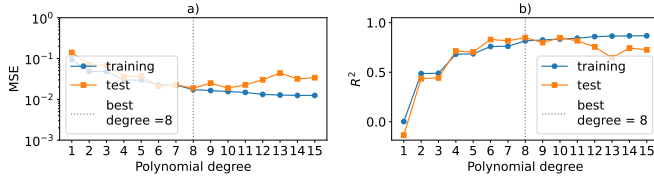


FIG. 1: In (a), the MSE is shown as a function of the polynomial degree, while in (b), the corresponding R^2 score is shown for OLS regression applied to the Runge function.

This indicates that the model explain approximately 85% of the variation of the test target values. Note that the training R^2 score for a degree 1 polynomial is substantially smaller (model explains $\approx 0.2\%$ of variation) than for polynomials of other tested degrees, see FIG. 1 b). The test R^2 score for degree 1 polynomial is negative, indicating that the model performs worse on the test data than the baseline predictor that always predicts the mean of the test target values. It is expected that a linear model would perform poorly on Runge’s function.

The applied scaling do change the numerical values of the coefficients θ_i . However, it does not change the fitted function or its predictions for OLS when the transformation is applied consistently. As a consequence, the scaling can be removed to increase the efficiency of the function when using the OLS. The scaling is nevertheless retained because the same function is used for other methods where scaling matters, such as Ridge and Lasso methods. Additionally, using the same preprocessing for different regression methods makes their performance easier to compare directly through the parameter values.

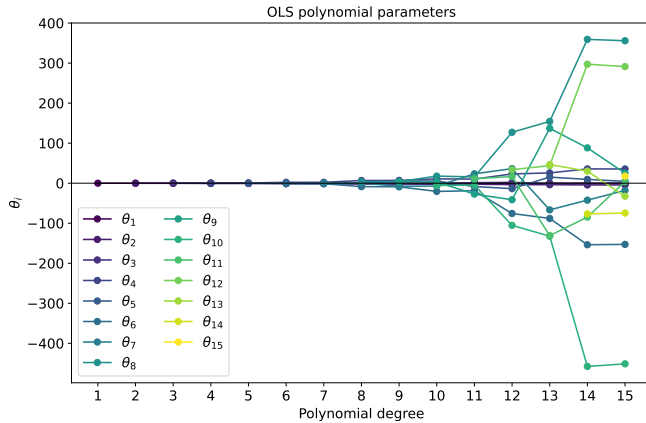


FIG. 2: The values of the elements of the parameter vector $\boldsymbol{\vartheta}$ are plotted as a function of the polynomial degree. Note that an element only appears once the corresponding degree has been reached.

The parameter plot, FIG. 2, shows how each parameter element ϑ_i changes as the polynomial degree is in-

creased, with ϑ_i first appearing when the polynomial degree reaches i . For low degrees, the elements are relatively small and stable. However, as the degree increases, especially around degrees 10-15 some elements become large in magnitude. These large parameter values of the OLS method (see FIG. 2) are associated with the polynomial features becoming strongly correlated, causing the design matrix to become ill-conditioned. This makes the fitted model increasingly sensitive to small changes in the data, and may lead to unstable polynomial fits at high degrees.

1. Bootstrap resampling and the bias-variance tradeoff for OLS

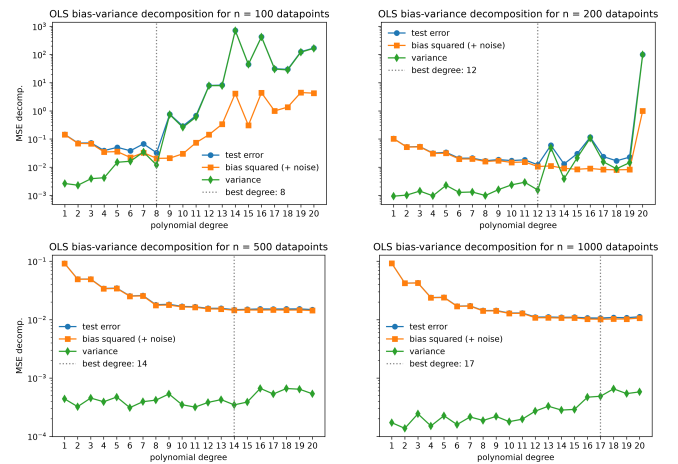


FIG. 3: Error decomposition of OLS after a bootstrap resampling with 100 bootstrap resamples on datasets with $n \in \{100, 200, 500, 1000\}$ data points. Note that the test error overlaps with the bias, and is therefore not visible.

In Figure 3, the error decomposition is shown for various sizes of datasets. The corresponding best degrees and test errors are displayed in Table II.

n	degree	test error
100	8	0.0325
200	12	0.0122
500	14	0.0148
1000	17	0.0106

TABLE II: The optimal degrees and test errors in bootstrap for different dataset sizes n .

It can generally be observed that the best polynomial degree becomes larger when dataset size is increased. This is to be expected as more data points allow for more statistics when making a fit and the model is thus not as dependent on each resample. We observe the smallest

test error for the largest dataset of $n = 1000$ with the highest polynomial degree.

As to be expected (see Section IC), we observe in all four cases that the variance (green) increases as the model becomes more complex (higher polynomial degree), though this effect is stronger when the dataset is smaller ($n \in \{100, 200\}$) with variance rising up to orders of magnitude of 10^2 and beyond at polynomial degrees beyond 10. This is likely due to the fact that with such small datasets, the higher order polynomials react too strongly to arbitrary fluctuations in the training datasets, yielding large fluctuations between resamples.

Inversely, up to polynomial degrees of approximately 10, the bias decreases for all sizes of datasets, and even beyond this degree for the larger datasets with $n \in \{200, 500, 1000\}$. For the smallest dataset, however, the bias rises again for polynomial degrees ≥ 9 , which is not what we would expect generally. Possible explanations for this observation could be that in this case, the number of data points in total is 100, meaning that the training dataset is very small with just 80 data points, from which the bootstrap is repeatedly sampling with replacement. It is therefore possible that in 100 bootstrap resamples, the subset that is randomly picked from the 80 data points is biased. While this is not necessarily limited to smaller dataset size, it is likely that the effect can be seen more clearly in this case. It would therefore be interesting to see if the phenomenon persists when the bootstrap resampling is performed more times.

Beyond this, the bias-variance tradeoff is visible quite well in all applications with the bias dominating the MSE for low model complexity, and the variance becoming increasingly significant at higher model complexity. The resulting optimal degrees vary quite a bit between the different dataset sizes, with the smallest dataset landing at the lowest optimal polynomial degree.

2. Cross-validation resampling for OLS

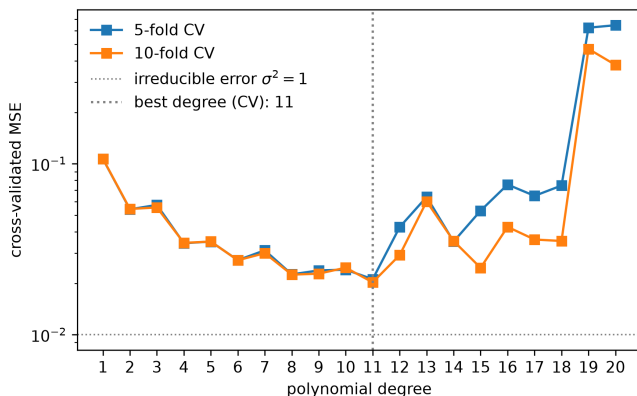


FIG. 4: k -fold cross validation for OLS with $k \in \{5, 10\}$ for a dataset with $n = 100$ data points.

In Figure 4, we see the MSE after 5- and 10-fold cross validation (CV) for polynomial degrees up to 20. The cross-validated MSE is smallest in both cases at the best polynomial degree of 11, with the MSE:s differing only very slightly. The exact values are given in Table III.

k	degree	cross-validated MSE
5	11	0.0211129
10	11	0.0202373

TABLE III: The optimal degrees and MSE in k -fold cross validation for a dataset of size $n = 100$.

In Figure 4, it is immediately apparent that at low model complexity, the number of folds did not make any significant difference for the cross-validated MSE. It is only after the optimal degree of 11 that the two curves start to look quite different, with the 5-fold version returning a larger MSE than the 10-fold CV.

It is also very interesting that for the same dataset size of $n = 100$, the previous bootstrapping method of Figure 3 yielded the smallest MSE for polynomial degree 8, while in cross validation we obtain best degree 11. Though not large, this difference could be due to the fact that in bootstrap resampling, the training data set, which is already quite small, is effectively reduced further by the resampling with replacement. In cross-validation, however, we are keeping out 20 data points as test data in the case of $k = 5$, and 10 data points in the case of $k = 10$, essentially leaving us with more data that can be fitted. Just like we already observe in the bootstrap resampling results for different dataset sizes, we end up with a slightly higher optimal polynomial degree for this larger dataset.

The MSE between the 5- and 10-fold CV differ only very slightly, with the 10-fold CV returning a slightly better result of 0.0202. This is likely due to the fact that for higher number of folds, the test folds become smaller and the training datasets larger, resulting in more training data overall.

B. Ridge regression

The lowest test MSE for the Ridge regression with degree 15 was obtained when the regularization parameter was $\lambda = 0.0398$, with the MSE 0.02562, the corresponding R^2 -value was 0.793, see FIG. 5 a). For comparison, the Ridge regression was also done for the optimal degree 8 found with plain OLS, see IV.

The Ridge method has slightly higher MSE, at 0.02562 compared to the optimal OLS model, which obtained an MSE of 0.0187, see FIG. 5 b). This difference is expected since the Ridge penalty introduces a bias by shrinking the regression parameters towards zero. Although this regularization can reduce the variance of the model and

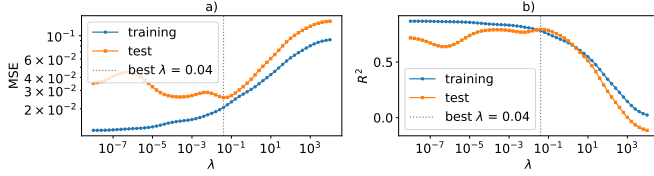


FIG. 5: Both plots show the Ridge regression of a degree 15 polynomial. In (a), the MSE is shown as a function of the regularization parameter λ , while in (b), the corresponding R^2 score is shown for Ridge regression applied to the Runge function.

make it less sensitive to the ill-conditioning of the design matrix, it can also lead to a small increase in bias. It can be inferred that at degree 15, the variance reduction from regularization does not sufficiently offset for the additional bias sufficiently to achieve a lower test MSE than the optimally chosen OLS model. However, when Ridge regression is used with the optimal degree 8 found with OLS, the MSE is reduced to 0.01765, representing an improvement over plain OLS. Additionally, there is a slight increase in the R^2 -score when using Ridge on the optimal degree; from 0.849 to 0.857. Therefore Ridge will provide an improvement of the models predictive performance compared to OLS. For an overview of the results, see table IV.

For $\lambda = 0$ the Ridge solution reduces to the OLS solution. This is illustrated when comparing the corresponding θ -parameters for Ridge and OLS in FIG. 2 and 6. When λ increases, the Ridge penalty increasingly suppresses the large parameter values. This is useful for higher-order polynomial features, which are the strongly correlated and can lead to an ill-conditioned design matrix. Increasing the value of the regularization parameter, therefore reduce the magnitude of the coefficients and make the model less sensitive to the ill-conditioning of the design matrix.

In FIG. 6 there are multiple larger parameter values when the regularization parameter is zero. As noted earlier, this corresponds to a ill-conditioned design matrix, and therefore with small singular values. Therefore, FIG. 6 illustrates how the shrinkage factor (6) suppress the SVD modes associated with the small singular values of a ill-conditioned design matrix.

The scaling of the polynomial features is essential in Ridge regression because the regularization term $\lambda \mathbf{I}$. Without scaling features of different magnitude would cause the regularization to have a different effect on the parameters. This would make the Ridge penalty dependent on the arbitrary scale of the features. Therefore, using the standard scaler ensure that λ has a more consistent effect on all parameters

Degree	λ	MSE	R^2
8	0	0.018712	0.849
8	0.00158	0.01765	0.857
15	0.0398	0.02562	0.793

TABLE IV: Results for OLS and Ridge regression. The first row corresponds to plain OLS with degree 8, while the second and third rows correspond to Ridge regression with polynomial degrees 8 and 15, respectively.

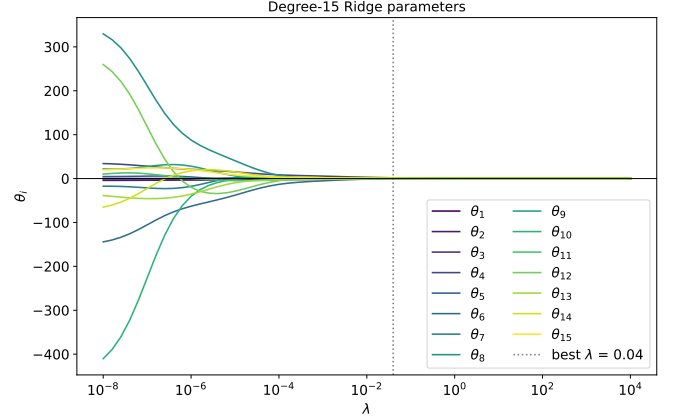


FIG. 6: The plot show the values of the elements of the parameter vector $\boldsymbol{\theta}$ for a degree 15-polynomial which are plotted as a function of the regularization parameter λ .

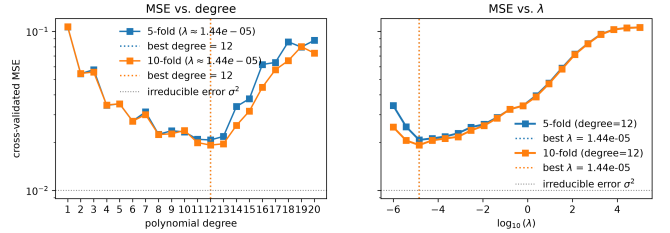


FIG. 7: Cross validation for Ridge: MSE vs. polynomial degree (left) and MSE vs. penalty parameter λ (right)

1. Cross-validation for Ridge regression

Figure 7 shows the results of a cross validation using Ridge regression with the MSE plotted against model complexity on the LHS and against the penalty parameter λ on the RHS. The MSEs obtained at optimal polynomial degree and λ for both $k = 5$ and $k = 10$ is displayed in Table V.

The cross-validation for Ridge regression returns the best degree of 12 for both $k = 5$ and $k = 10$, and also the best penalty parameters λ are identical for the two runs. The cross-validated MSE is slightly smaller for the 10-fold CV, which is likely again due to the larger training datasets that can be used for higher fold number k .

Further, we observe quite a large difference between the

k	degree	$\lambda[10^{-5}]$	cross-validated MSE
5	12	1.438	0.0207671
10	12	1.438	0.0192712

TABLE V: The optimal degrees and MSE in k -fold cross validation for a dataset of size $n = 100$.

best penalty parameter λ that is returned when doing a simple Ridge regression analysis as in Figure 5, versus the values obtained after cross validation. This can be explained by two different factors: on the one hand, the cross validation returns not a sole best λ , but the best combination of polynomial degree and λ . With the plain Ridge regression performed at degree 15, it is natural that a larger λ is needed to keep the parameters $\boldsymbol{\vartheta}_i$ well-balanced. At the lower polynomial degree of 12 that is returned by both 5- and 10-fold CV, a smaller λ already does the job.

Moreover, by doing the cross-validation, the cross-validated MSE is averaged over k differing datasets, meaning our model is less influenced by how the training/ testing data split is made. It would therefore be interesting to see in a next step how λ changes in plain Ridge regression with varying choices of test/ training data to confirm or deny this hypothesis.

C. Plain gradient descent

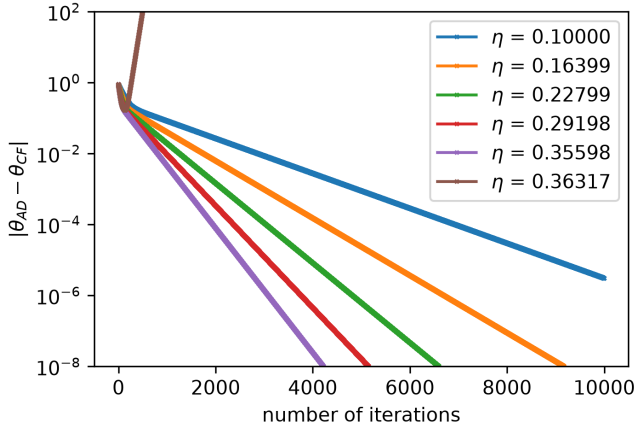


FIG. 8: Gradient descent for OLS for different learning rates η

η	N
0.10000	$> 10^4$
0.16399	9170
0.22799	6594
0.29198	5147
$0.99\eta_{\max} = 0.35598$	4220
$1.01\eta_{\max} = 0.36317$	—

TABLE VI: Values of the learning rate η and the number N of iterations needed for convergence to reach tolerance $t = 10^{-8}$

Figure 8 shows comparison of the parameters $\boldsymbol{\vartheta}$ obtained by the plain gradient with the closed form parameters obtained by pure OLS. Their absolute difference is on the y -axis of the plot. Having chosen the learning rate values η in an evenly spaced array of values between 0.1 and $0.99\eta_{\max}$, we see all of the curves converge, though not at the same rates. The exact iteration numbers needed to reach the set tolerance of 10^{-8} are in Table VI. There, it becomes clear that as η approaches its maximum value, tolerance is reached faster with iteration numbers needed roughly cut into half when using $0.99\eta_{\max}$ with respect to the numbers needed for the smaller learning rates η . For the smallest learning rate $\eta = 0.1$, tolerance is not reached within the fixed iterations of 10^4 .

Moreover, it is interesting to study the learning rates that are right at the border of convergence. While we $\eta = 0.99\eta_{\max}$ reach tolerance see relatively fast, using a learning rate that is only a little bit beyond this value explodes after only a few iterations. This is visible for the largest value of $\eta = 1.01\eta_{\max}$ in Figure 8.

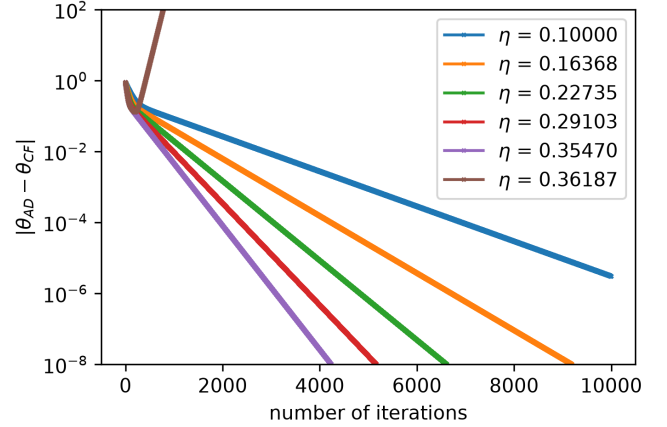


FIG. 9: Gradient descent for Ridge for different learning rates η with the Ridge penalty parameter set to $\lambda = 1.438 \cdot 10^{-5}$

η	N
0.10000	$> 10^4$
0.16368	9188
0.22735	6612
0.29103	5164
$0.99\eta_{max} = 0.35470$	4235
$1.01\eta_{max} = 0.36187$	-

TABLE VII: Values of the learning rate η and the number of iterations needed for convergence to reach tolerance $t = 10^{-8}$. The Ridge penalty parameter was set to $\lambda = 1.438 \cdot 10^{-5}$

The analogous analysis for Ridge regression is shown in Figure 9 and the exact numbers for the iterations needed in Table VII. We observe the analogous phenomena to what already happened for the gradient descent applied to OLS, with only the constant η_{max} varying slightly due to the modification to the Hessian matrix with the Ridge penalty parameter λ .

D. Momentum and adaptive gradient methods

TABLE VIII: The fewest updates needed for each model to reach the target. A dash means the target was not reached within 20,000 updates.

Model	OLS Degree 5	Ridge Degree 5	OLS Degree 8	Ridge Degree 8	OLS Degree 15	Ridge Degree 15
Plain	566	42	—	118	—	131
Momentum	73	64	9776	70	—	79
AdaGrad	958	44	—	87	—	212
RMSProp	481	40	—	73	—	146
Adam	171	39	8424	71	—	78

From Table VIII we can read that for degree-5 OLS, momentum reaches the target in 73 updates at rate 0.3162, compared with 17 897 at rate 0.001. Degree-8 OLS reaches the target only with momentum and Adam, their fastest runs require 9 776 and 8 424 updates. No degree-15 OLS run recorded to reach the target within the budget of updates. Every method succeeds for at least one Ridge rate at each tested degree.

The degree-15 shows budget exhaustion and from instability. Plain descent at rate 0.1 ends at 20 000 updates with gap 0.074098; rate 0.316228, 1.0 stops as unstable after 10, 6 updates respectively. For momentum at rates 0.3612, 1.0 shows instability after 12, 6 updates respectively. While budget exhaustion was recorded for the rest of methods. The complete recorded statuses appear in Table XVII. The behavior of OLS for different degrees is presented in Figure 10, 11, 12.

Figure 10 shows that the number of updates depends on both the method and the learning rate. For OLS, the fastest tested momentum run reaches the target in 73 updates, compared with 171 for Adam and 566 for plain gradient descent. For Ridge, Adam reaches the target in 39 updates, followed closely by RMSProp with 40 and plain gradient descent with 42. These minima occur at different learning rates. Smaller

rates often require many more updates, and some rates do not reach the target within the limit.

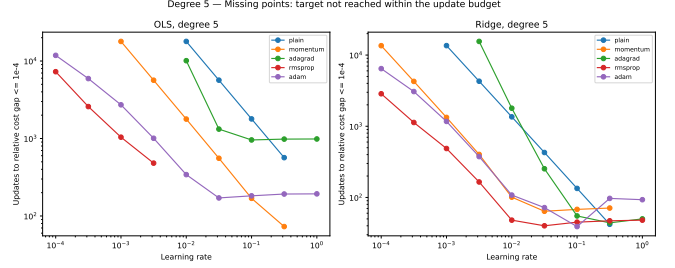


FIG. 10: updates needed to first reach relative cost gap 10^{-4} for degree-5 OLS and Ridge. Only runs reaching the target within 20 000 updates are plotted. Missing points for an optimizer indicate that the target was not reached within the update limit. For Ridge, λ was set to $\lambda = 0.1$.

In Figure 11, only momentum and Adam reach the target for OLS at the tested rates. Their fastest runs require 9 776 and 8 424 updates, respectively. Ridge reaches the target with all five methods at suitable rates. Its lowest counts are 70 for momentum, 71 for Adam, 73 for RMSProp, 87 for AdaGrad and 118 for plain gradient descent. Compared with degree 5, OLS needs substantially more updates, while Ridge still reaches the target in relatively few updates.

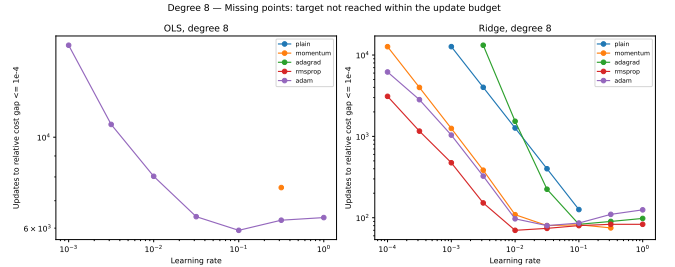


FIG. 11: updates needed to first reach relative cost gap 10^{-4} for degree-8 OLS and Ridge. Only runs reaching the target within 20,000 updates are plotted. Missing points for an optimizer indicate that the target was not reached within the update limit. For Ridge, λ was set to $\lambda = 0.1$.

Figure 12 shows a clear difference between OLS and Ridge. No OLS run reaches the target within 20 000 updates, so the OLS panel has no plotted points. Ridge does reach the target with all five methods at suitable rates. The fastest runs require 78 updates for Adam, 79 for momentum, 131 for plain gradient descent, 146 for RMSProp and 212 for AdaGrad. Thus, the absence of successful runs applies to OLS, not to Ridge.

Figures 10–12 and Table VIII show that no update method needs the fewest updates in every case. For degree-5 OLS, momentum has the lowest count among the five methods: 73 updates. For degree-8 OLS, only momentum and Adam reach the target. Each count is the lowest successful count across the tested learning rates, so these comparisons do not necessarily use the same rate.

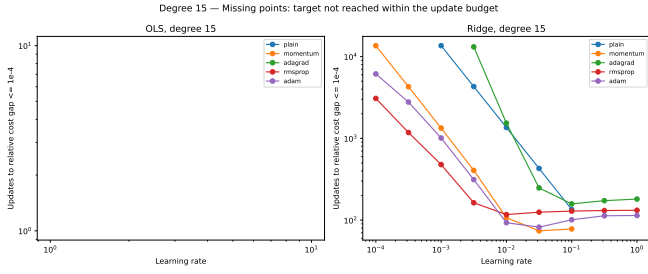


FIG. 12: Updates needed to first reach relative cost gap 10^{-4} for degree-15 OLS and Ridge. Only runs reaching the target within 2000 updates are plotted. No reaching of the target was detected within the limit. For Ridge, λ was set to $\lambda = 0.1$.

OLS becomes harder to optimize as the degree increases: none of the tested methods reaches the target for degree 15. Ridge reaches the target with all five methods at suitable rates for all degrees. This is consistent with its penalty improving the conditioning of the problem, although conditioning was not directly measured here. The detailed rate results also show why choosing the learning rate matters. Small rates can give slow progress, while large rates can cause instability. The optimization results should be distinguished from prediction accuracy, as can be seen in Table IX. Ridge has a smaller final relative cost gap than OLS, but a higher test MSE. Ridge is therefore closer to its own training-objective minimum, while OLS predicts the test observations more accurately in this comparison.

E. Lasso regression

The saved JAX absolute-value check returns 1 at zero, -1 at -0.3 , and 1 at 0.3 . The zero result is a valid subgradient, although the own solver uses zero instead. Selected degree-8 results below illustrate the difference between fitting the objective and predicting the test observations.

TABLE IX: Degree-8 comparison from the saved training-gap-selected summary. Penalties are mean-cost lambda.

Model	Penalty	Update method	Final gap	Test MSE
OLS	0.0	Adam	9.959e-05	0.018466
Ridge	0.1	Adam	3.614e-05	0.057772
Lasso	0.001	Adam	8.831e-05	0.022916
Lasso	0.01	Adam	8.628e-05	0.039754
Lasso	0.1	Adam	9.808e-05	0.075580

From Tables XVIII–XIX–XX, we can summarize table IX, at degree 8 and $\lambda = 0.001$, momentum and Adam reach the target after 1,781 and 1,850 updates in the printed selections. Plain descent, AdaGrad and RMSProp remain above it at 20,000 updates. At degree 15 and $\lambda = 0.1$, all five selected gaps remain above the target.

Figure 13 shows for $\lambda = 0.001$, the figure shows that reaching the Lasso target becomes more difficult for several methods as

the degree increases. At degree 5, all five methods reach the target at a suitable rate. At degrees 8 and 15, only momentum and Adam reach it. At degree 8, the selected plain, AdaGrad and RMSProp runs finish with gaps of 2.005×10^{-3} , 1.102×10^{-3} and 8.676×10^{-4} , respectively, all above the 10^{-4} target. The successful learning-rate choices therefore depend on both the method and the degree.

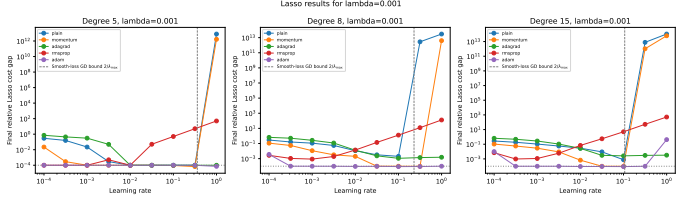


FIG. 13: Final relative Lasso cost gap for $\lambda = 0.001$. The horizontal line marks the target. The vertical line marks $2/\lambda_{\max}(H)$, where $\lambda_{\max}(H)$ is the largest eigenvalue of the squared-error Hessian. This is a reference bound for plain gradient descent on the smooth loss, not a guaranteed stability boundary for Lasso or adaptive updates.

In Figure 14 for $\lambda = 0.01$, all five methods reach the target at some rate for each of the three degrees. This differs from the $\lambda = 0.001$ results, where only momentum and Adam reach it at degrees 8 and 15. But, the successful rates are not the same for every method. At degree 15, for example, the printed selections use rates of 0.01 for plain gradient descent, approximately 0.003162 for momentum and 0.0001 for RMSProp.

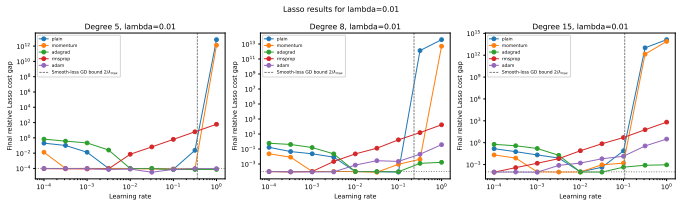


FIG. 14: Final relative Lasso cost gap for $\lambda = 0.01$. The horizontal line marks the target. The vertical line marks $2/\lambda_{\max}(H)$, where $\lambda_{\max}(H)$ is the largest eigenvalue of the squared-error Hessian. This is a reference bound for plain gradient descent on the smooth loss, not a guaranteed stability boundary for Lasso or adaptive updates.

Figure 15 shows that the larger penalty $\lambda = 0.1$ does not make every run reach the target. All five methods succeed at a tested rate for degree 5. At degree 8, RMSProp remains above the target even at its printed gap is 1.603×10^{-3} , while the other four methods reach it. At degree 15, none of the methods reaches the target within the budget. AdaGrad has printed gap 3.682×10^{-4} , which is still above 10^{-4} .

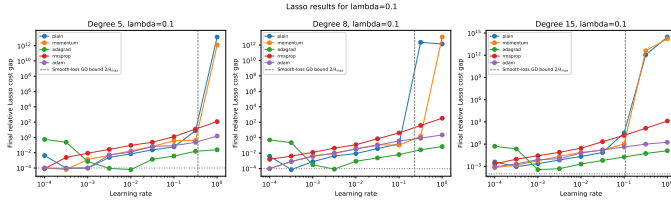


FIG. 15: Final relative Lasso cost gap for $\lambda = 0.1$. The horizontal line marks the target. The vertical line marks $2/\lambda_{\max}(H)$, where $\lambda_{\max}(H)$ is the largest eigenvalue of the squared-error Hessian. This is a reference bound for plain gradient descent on the smooth loss, not a guaranteed stability boundary for Lasso or adaptive updates.

Conclusively, Figures 13, 14, 15 show that convergence is affected by degree, penalty, and learning rate. A stronger penalty does not always make the target easier to reach. The absolute-value penalty is not differentiable at zero, as shown in Eq. (3.58) in [2], which makes its optimization different from OLS and Ridge. The vertical reference line marks $2/\lambda_{\max}(H)$ in these figures and represents a learning-rate limit for plain gradient descent on the squared-error cost without the Lasso penalty. The line is included only for comparison. It does not guarantee that Lasso or the adaptive methods will converge at rates below it.

Table IX shows that all three selected **Lasso** runs reach the target, but their test errors increase with the penalty, from 0.022916 for $\lambda = 0.001$ to 0.075580 for $\lambda = 0.1$. A small training-cost gap therefore does not necessarily mean good prediction accuracy.

F. Stochastic gradient descent

All runs use degree 8 and 1 000 epochs. batch sizes 5, 20 and 80 give 16 000, 4 000, 1 000 updates, respectively. Batch 80 represents full-batch gradient descent. Table XXI gives a comparison of all methods and both schedules.

No OLS or Lasso run has a final gap below the target of 10^{-4} . The smallest OLS gap is 0.09013 for full-batch Adam with a constant learning rate, while the smallest **Lasso** gap is 0.003577 under the same setting.

Ridge has several runs below the target. For example, batch-20 Adam with a decreasing learning rate gives a gap of 5.805×10^{-7} , while full-batch Adam with a constant learning rate gives 1.102×10^{-8} . Some Ridge runs, such as full-batch momentum with constant steps and full-batch Adam with decreasing steps reach the target with gaps that are printed as zero. These zeros represent numerical agreement with the reference.

Figure 16 shows the results for the smallest batch with a constant learning rate. After 1 000 epochs, none of the **OLS** or **Lasso** methods has a gap below 10^{-4} . For **Ridge**, only AdaGrad finish below the target, with gaps of 3.126×10^{-5} . Momentum finishes with much larger gaps: 0.9668 for OLS, 5.989 for Ridge and 1.278 for Lasso.

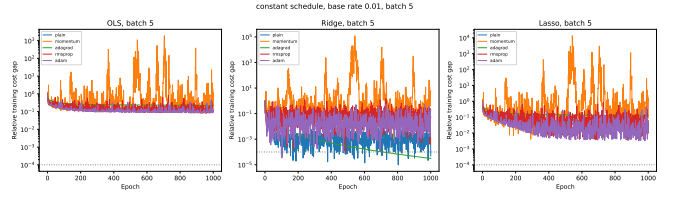


FIG. 16: constant schedule, batch size 5, base rate 0.01. Curves show relative objective gap versus epoch.

With batch size 20 and constant steps, Figure 17 gives smaller final momentum gaps than batch size 5 for all three models. They are 0.09347 for **OLS**, 0.001378 for **Ridge** and 0.005486 for **Lasso**. **Momentum** gives the smallest final **OLS** and **Lasso** gaps among the five methods in this setting, but neither reaches the target. For **Ridge**, **plain gradient descent** and **AdaGrad** finish below the target, with gaps of 3.922×10^{-6} and 7.018×10^{-5} respectively.

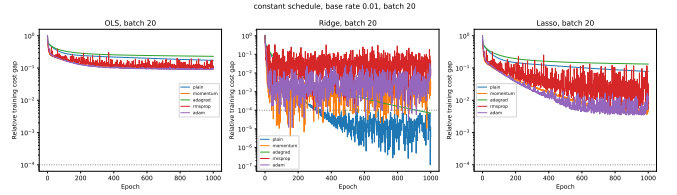


FIG. 17: constant schedule, batch 20, base rate 0.01. Curves show relative objective gap versus epoch.

Figure 18 uses all 80 training observations in every update (the full dataset), providing a comparison without stochastic gradient descent. It is therefore that the curves differ quite significantly from the ones with smaller batch sizes. **Adam** gives the smallest final gap for **OLS**, 0.09013, and **Lasso**, 0.003577, but both remain above the target. For **Ridge**, **momentum** prints a zero gap and **Adam** finish at 1.102×10^{-8} . The printed zero indicates agreement at numerical precision. These full-batch runs use 1 000 updates over 1 000 epochs, compared with 4 000 updates for batch 20 and 16 000 for batch 5.

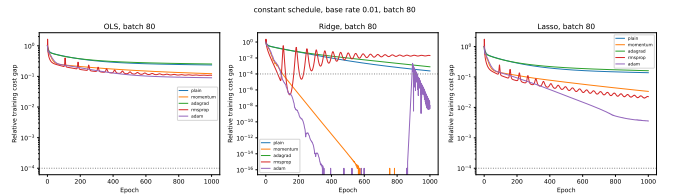


FIG. 18: constant schedule, batch size 80, base rate 0.01. Curves show relative objective gap versus epoch. Batch 80 is the full-batch baseline.

Compared with the constant schedule at batch size 5, Figure 19 shows a much better final result for **momentum** in all three models. Its gaps fall to 0.1584 for **OLS**, 4.994×10^{-7} for **Ridge** and 0.06545 for **Lasso**. **Ridge** also finishes below the target with **RMSProp** and **Adam**. However, decreasing steps do not improve every method: **Ridge**, **AdaGrad** fin-

ishes at 0.1097. **OLS** and **Lasso** remain above the target for every method.

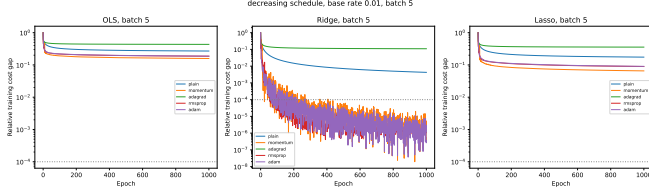


FIG. 19: decreasing schedule, batch size 5, base rate 0.01. Curves show relative objective gap versus epoch.

In Figure 20, **Ridge** reaches a final gap below the target with **momentum**, **RMSProp** and **Adam**. Their gaps are 3.385×10^{-7} , 1.286×10^{-6} and 5.805×10^{-7} , respectively. Plain gradient descent and AdaGrad remain above the target. For **OLS** and **Lasso**, every method finishes with a larger gap than its corresponding constant-schedule run at batch size 20. Thus, this decreasing schedule improves some **Ridge** results but gives no improvement for **OLS** or **Lasso** at this batch size.

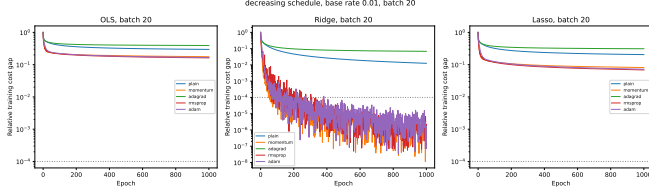


FIG. 20: decreasing schedule, batch size 20, base rate 0.01. Curves show relative objective gap versus epoch.

Figure 21 combines decreasing steps with the full training batch. For **Ridge**: **momentum**, **RMSProp** and **Adam** finish below the target, with Adam printing a zero gap. For **OLS** and **Lasso**, **RMSProp** improves slightly compared with its constant-schedule result, but the other four methods finish with larger gaps. Neither **OLS** nor **Lasso** reaches the target. In particular, Lasso Adam finishes at 0.05535, compared with 0.003577 with constant steps.

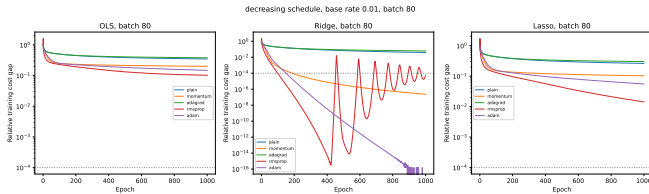


FIG. 21: Decreasing schedule, batch size 80, base rate 0.01. Curves show relative objective gap versus epoch. Batch size 80 is the full-batch baseline.

G. Final Model Selection

Table X shows the lowest cross-validation MSE for each method using five and ten folds. Ridge with polynomial de-

gree 12 gives the lowest error in both fold comparisons.

Both comparisons select approximately $\alpha = 1.43845 \times 10^{-5}$, where α is the Ridge penalty used by Scikit-Learn. Our Ridge cost uses the mean squared error, while Scikit-Learn uses the sum of squared errors. To express the same penalty in our cost function, we use $\lambda = \alpha/n_{\text{train}}$.

Five-fold cross-validation uses 80 training observations per fold, giving $\lambda \approx 1.79806 \times 10^{-7}$. Ten-fold cross-validation uses 90 training observations per fold, giving $\lambda \approx 1.59828 \times 10^{-7}$.

TABLE X: Selected configurations for each method and fold count. Penalties follow the mean-cost convention.

Folds	Method	Degree	Mean-cost λ	CV MSE
5	OLS	11	-	0.0211129
5	Ridge	12	$1.79806\text{e-}07$	0.0207671
5	Lasso	11	$1\text{e-}06$	0.0211418
10	OLS	11	-	0.0202373
10	Ridge	12	$1.59828\text{e-}07$	0.0192712
10	Lasso	13	$1.43845\text{e-}05$	0.0201081

With five-fold cross-validation, the lowest errors for OLS and Lasso occur at polynomial degree 11. With ten folds, OLS still performs best at degree 11, while Lasso performs best at degree 13. For Lasso with five folds, the best penalty is the smallest one we tested. This means we cannot tell whether an even smaller penalty would improve the result. For Lasso with ten folds and Ridge with either fold count, the best penalty lies between the smallest and largest tested values. These errors come from the same cross-validation runs used to choose the models. We have not evaluated the selected models on a separate test set that was kept out of the selection process.

Figure 22 shows that the cross-validation error falls strongly as the degree increases from the lowest values. At higher degrees, **OLS** has several increases in error, while the **Ridge** and **Lasso** curves change less. **Ridge** at degree 12 gives the lowest error in both fold comparisons: 0.0207671 with five folds and 0.0192712 with ten folds. **OLS** selects degree 11 in both folds. **Lasso** selects degree 11 with five folds and degree 13 with ten folds.

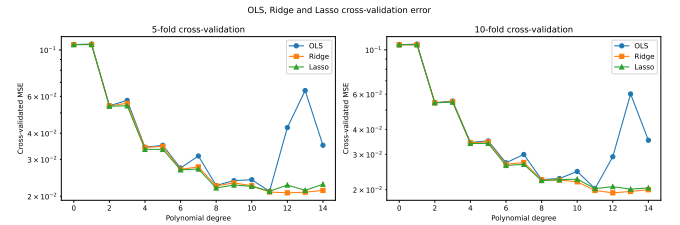


FIG. 22: CV MSE versus degree for five and ten folds. Ridge and Lasso use the lowest tested error over penalties at each degree

Figure 23 shows how the error changes with the penalty when the degree is held at its selected value. **Ridge** has its lowest error at $\lambda \approx 1.79806 \times 10^{-7}$ with five folds and $\lambda \approx 1.59828 \times 10^{-7}$ with ten folds. **Lasso** selects 10^{-6} with five folds and approximately 1.43845×10^{-5} with ten folds. The five-fold Lasso choice is the smallest penalty tested. At

large penalties, the errors rise and become nearly constant, well above their minimum values.

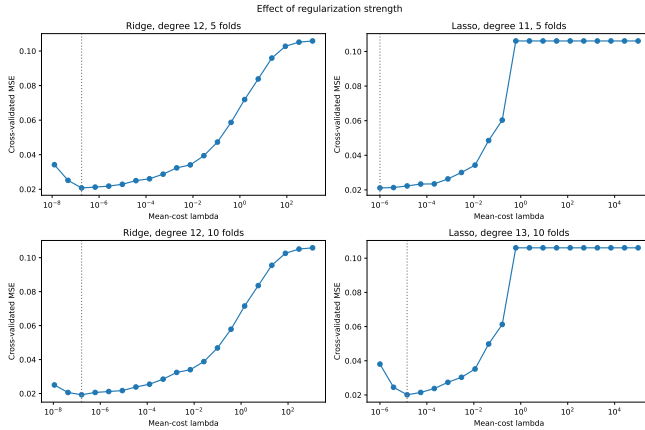


FIG. 23: CV MSE versus penalty at the selected degree for each method and fold count. The vertical line marks the selected penalty.

The bias-variance decomposition helps interpret these comparisons. Increasing the polynomial degree allows **OLS** to fit more complex curves, which can reduce bias but increase variance. **Ridge** and **Lasso** restrict the coefficients to control variance, at the possible cost of increased bias. The lower selected Ridge errors are consistent with the effect of regularization in this experiment. The final model selection reports cross-validation MSE, not separate bias and variance values, so it does not show how much each term changes.

III. CONCLUSION

We studied how the choice of optimization method and regularization affects polynomial regression for Runge’s function. For degree-5 OLS and Ridge, momentum and Adam, respectively, needed the fewest updates among the tested methods and learning rates to reach a relative training-cost gap of 10^{-4} : 73 updates for OLS and 39 for Ridge. The comparison showed that no update method needed the fewest updates in every case: for degree-8 OLS, Adam needed 8 424 updates compared with 9 776 for momentum, with the learning rate playing an important role in convergence. Our Lasso implementation reached a relative cost gap below 10^{-4} for degree 8 with Adam at penalties of 0.001, 0.01 and 0.1, but the corresponding test MSE increased from 0.022916 to 0.039754 and 0.075580. Compared with updates using the full training set, stochastic gradient descent did not consistently improve the final cost gap: after 1 000 epochs, full-batch Adam with constant steps gave the smallest recorded OLS and Lasso gaps, 0.09013 and 0.003577, although neither reached the 10^{-4} target. The final cross-validation analysis selected Ridge at polynomial degree 12, with a mean MSE of 0.0207671 using five folds and 0.0192712 using ten folds. This suggests that Ridge was

the best choice for prediction among the tested models and settings, while reaching a small training-cost gap alone did not guarantee a low test error.

These comparisons have limitations. The results apply to Runge’s function and the data sizes, noise levels, and parameter settings tested. They do not establish that one optimizer is always better than the others. A method may appear less accurate because its learning rate is poorly chosen or because it has not been given enough iterations to converge. Iteration counts also need care when comparing full data and mini batch updates since each update uses a different amount of data. For Lasso, the penalty is not differentiable at zero, so its treatment must be considered when interpreting the fitted coefficients. Comparisons between implementations also depend on using consistent definitions of the cost function and penalty strength.

Two useful next steps would be: first, to examine learning rates more closely around the best settings, which would help determine if differences between optimizers remain after further adjustment, and second, to test a stopping rule for stochastic gradient descent based on validation error, which would help evaluate if training can stop earlier without reducing prediction accuracy.

Appendix A: Use of AI/LLM Tools

A.1 AI Tools used

AI Tools used:

- GPT-5.6 Sol(September 2026),
- GitHub Copilot (VS Code, throughout)
- Claude Sonnet 5.5 Browser Version

A.2 Text writing and editing

The levels of AI use for text are defined as follows:

- **0 – None:** No AI assistance was used.
- **1 – Language only:** AI assistance was limited to grammar, spelling, and minor phrasing corrections.
- **2 – Editorial:** The model rewrote or restructured sentences, while the ideas were supplied by the author and the result was checked by the author.
- **3 – Generative:** The model drafted substantial text based on the author’s prompts. The author subsequently revised and verified the text and takes full responsibility for its content.

TABLE XI: AI declaration text

Section	Level	Notes
Abstract	2	Was given a written text, then it was checked for mistakes and condensed. It also helped with restructuring some sentences.
Introduction	2	it checked for mistakes and made the given text more coherent
Methods	2	Used as a fact checker and challenger.
Results	2	It was used as a fact checker and challenger of discussions that were already written. Some corrections that came from this were implemented in our own words (and understanding).
Conclusion	2	it helped to correctly locate the "[...]" and refill them later when we get the results.
Formatting	?	occasional use throughout to format

A.3 Code generation and assistance

The levels of AI use for code assistance are defined as follows:

- 0 – **None:** No AI assistance was used.

- 1 – **Debugging:** The model identified a bug, you fixed it.
- 2 – **Snippet:** A function, loop or short block that you integrated and tested.
- 3 – **Skeleton:** The model generated the structure, you filled in the logic
- 4 – **Substantial:** The model wrote most of the file, you adapted, tested and commented it.

TABLE XII: AI declaration code generation and assistance

Section	Level	Notes
Code, tests, implementation	2	Helped with marked snippets of code, particularly for plotting, and debugging
parts c), d) and e) in the notebook, except for plotting	1,2	AI used for debugging and filling in correct syntax/indices in handwritten loops and functions
parts c), d) and e) all plotting code	4	Entirely generated, then checked if the correct arrays were plotted and made some small occasional changes.
...	...	...

-
- [1] Wikipedia contributors. Runge’s phenomenon. https://en.wikipedia.org/wiki/Runge%27s_phenomenon, 2026. Wikipedia, The Free Encyclopedia. Accessed: 2026-10-06.
- [2] Morten Hjorth-Jensen. *Machine Learning and the Physical Sciences*. Springer, 2026.
- [3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer New York, New York, NY, 2009.
- [4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [5] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [6] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

TABLE XIII: Number of completed method/rate experiments in part f.

Model	Degree	Runs
OLS	5	45
Ridge	5	45
OLS	8	45
Ridge	8	45
OLS	15	45
Ridge	15	45

Appendix B: Additional Results

1. **Part f**
2. **Part g**
3. **Part h**
4. **Part i**

TABLE XIV: target-hit counts for degree 5. A dash means the target was not reached within 20,000 updates.

Model	Rate	Plain	Momentum	AdaGrad	RMSProp	Adam
OLS	0.0001	—	—	—	7281	11833
OLS	0.0003162	—	—	—	2582	5934
OLS	0.001	—	17897	—	1040	2722
OLS	0.003162	—	5653	—	481	1007
OLS	0.01	17907	1781	10100	—	342
OLS	0.03162	5662	556	1322	—	171
OLS	0.1	1790	169	958	—	182
OLS	0.3162	566	73	982	—	192
OLS	1	—	—	986	—	193
Ridge	0.0001	—	13547	—	2861	6450
Ridge	0.0003162	—	4267	—	1130	3073
Ridge	0.001	13572	1331	—	487	1170
Ridge	0.003162	4291	402	15550	165	375
Ridge	0.01	1356	101	1792	48	108
Ridge	0.03162	428	64	253	40	72
Ridge	0.1	134	68	55	45	39
Ridge	0.3162	42	71	44	47	97
Ridge	1	—	—	50	48	93

TABLE XV: target-hit counts for degree 8. A dash means the target was not reached within 20,000 updates.

Model	Rate	Plain	Momentum	AdaGrad	RMSProp	Adam
OLS	0.0001	—	—	—	—	—
OLS	0.0003162	—	—	—	—	—
OLS	0.001	—	—	—	—	19347
OLS	0.003162	—	—	—	—	13010
OLS	0.01	—	—	—	—	10264
OLS	0.03162	—	—	—	—	8782
OLS	0.1	—	—	—	—	8424
OLS	0.3162	—	9776	—	—	8592
OLS	1	—	—	—	—	8675
Ridge	0.0001	—	11898	—	2996	6041
Ridge	0.0003162	—	3751	—	1126	2741
Ridge	0.001	11914	1174	—	449	999
Ridge	0.003162	3767	360	12102	145	311
Ridge	0.01	1191	103	1419	73	93
Ridge	0.03162	376	70	210	88	71
Ridge	0.1	118	78	87	104	81
Ridge	0.3162	—	76	109	121	90
Ridge	1	—	—	136	—	115

TABLE XVI: target-hit counts for degree 15. A dash means the target was not reached within 20,000 updates.

Model	Rate	Plain	Momentum	AdaGrad	RMSProp	Adam
OLS	0.0001	—	—	—	—	—
OLS	0.0003162	—	—	—	—	—
OLS	0.001	—	—	—	—	—
OLS	0.003162	—	—	—	—	—
OLS	0.01	—	—	—	—	—
OLS	0.03162	—	—	—	—	—
OLS	0.1	—	—	—	—	—
OLS	0.3162	—	—	—	—	—
OLS	1	—	—	—	—	—
Ridge	0.0001	—	13150	—	2951	5919
Ridge	0.0003162	—	4144	—	1159	2642
Ridge	0.001	13169	1296	—	—	947
Ridge	0.003162	4164	395	11620	180	294
Ridge	0.01	1316	107	1377	146	88
Ridge	0.03162	415	85	244	202	78
Ridge	0.1	131	79	212	229	99
Ridge	0.3162	—	—	283	—	123
Ridge	1	—	—	362	—	137

TABLE XVII: Complete saved degree-15 OLS diagnostics. Budget means the update limit was reached; unstable means the numerical guard stopped the run.

Method	Rate	Status	Updates	Final gap
Plain	0.0001	budget	20000	3.9148e-01
Plain	0.0003162	budget	20000	3.1487e-01
Plain	0.001	budget	20000	2.3896e-01
Plain	0.003162	budget	20000	1.9055e-01
Plain	0.01	budget	20000	1.5358e-01
Plain	0.03162	budget	20000	1.0194e-01
Plain	0.1	budget	20000	7.4098e-02
Plain	0.3162	unstable	10	6.5004e+12
Plain	1	unstable	6	8.8367e+13
Momentum	0.0001	budget	20000	2.3897e-01
Momentum	0.0003162	budget	20000	1.9055e-01
Momentum	0.001	budget	20000	1.5359e-01
Momentum	0.003162	budget	20000	1.0194e-01
Momentum	0.01	budget	20000	7.4100e-02
Momentum	0.03162	budget	20000	5.8117e-02
Momentum	0.1	budget	20000	4.2361e-02
Momentum	0.3162	unstable	12	1.2387e+13
Momentum	1	unstable	6	5.0893e+13
AdaGrad	0.0001	budget	20000	7.0600e-01
AdaGrad	0.0003162	budget	20000	5.5934e-01
AdaGrad	0.001	budget	20000	3.8475e-01
AdaGrad	0.003162	budget	20000	2.3272e-01
AdaGrad	0.01	budget	20000	1.3592e-01
AdaGrad	0.03162	budget	20000	8.2528e-02
AdaGrad	0.1	budget	20000	7.9694e-02
AdaGrad	0.3162	budget	20000	8.1158e-02
AdaGrad	1	budget	20000	8.3183e-02
RMSProp	0.0001	budget	20000	1.0097e-01
RMSProp	0.0003162	budget	20000	7.5799e-02
RMSProp	0.001	budget	20000	7.3456e-02
RMSProp	0.003162	budget	20000	7.6720e-02
RMSProp	0.01	budget	20000	1.1485e-01
RMSProp	0.03162	budget	20000	4.9795e-01
RMSProp	0.1	budget	20000	4.3278e+00
RMSProp	0.3162	budget	20000	4.2623e+01
RMSProp	1	budget	20000	4.2556e+02
Adam	0.0001	budget	20000	1.0357e-01
Adam	0.0003162	budget	20000	5.4047e-02
Adam	0.001	budget	20000	4.0480e-02
Adam	0.003162	budget	20000	3.8504e-02
Adam	0.01	budget	20000	3.7971e-02
Adam	0.03162	budget	20000	3.7828e-02
Adam	0.1	budget	20000	3.7811e-02
Adam	0.3162	budget	20000	3.7777e-02
Adam	1	budget	20000	3.7611e-02

TABLE XVIII: Degree-5 selections by lowest final training cost gap. Test MSE does not select the rate.

Model	λ	Method	Rate	Updates	Gap	Test MSE
OLS	0.0	Plain	0.3162	566	9.947e-05	0.036359
OLS	0.0	Momentum	0.3162	73	9.637e-05	0.036483
OLS	0.0	AdaGrad	0.03162	1322	9.953e-05	0.036715
OLS	0.0	RMSProp	0.003162	481	9.788e-05	0.036497
OLS	0.0	Adam	0.03162	171	9.663e-05	0.036352
Ridge	0.1	Plain	0.3162	42	8.439e-05	0.064000
Ridge	0.1	Momentum	0.03162	64	2.464e-05	0.063414
Ridge	0.1	AdaGrad	0.1	55	8.974e-05	0.063991
Ridge	0.1	RMSProp	0.1	45	9.332e-05	0.063972
Ridge	0.1	Adam	0.03162	72	2.192e-05	0.063695
Lasso	0.001	Plain	0.3162	327	9.966e-05	0.035646
Lasso	0.001	Momentum	0.3162	55	6.688e-05	0.035768
Lasso	0.001	AdaGrad	0.03162	1300	9.963e-05	0.036023
Lasso	0.001	RMSProp	0.0003162	2542	9.977e-05	0.036066
Lasso	0.001	Adam	1	118	7.222e-05	0.035680
Lasso	0.01	Plain	0.1	577	7.705e-05	0.039758
Lasso	0.01	Momentum	0.03162	139	8.759e-05	0.039712
Lasso	0.01	AdaGrad	0.1	266	7.244e-05	0.039671
Lasso	0.01	RMSProp	0.001	917	8.688e-05	0.039773
Lasso	0.01	Adam	0.03162	184	3.209e-05	0.039462
Lasso	0.1	Plain	0.001	7226	9.786e-05	0.075575
Lasso	0.1	Momentum	0.0003162	3911	6.412e-05	0.075573
Lasso	0.1	AdaGrad	0.01	5872	6.255e-05	0.075577
Lasso	0.1	RMSProp	0.0001	2524	8.226e-05	0.075577
Lasso	0.1	Adam	0.0003162	4006	9.009e-05	0.075572

TABLE XIX: Degree-8 selections by lowest final training cost gap. Test MSE does not select the rate.

Model	λ	Method	Rate	Updates	Gap	Test MSE
OLS	0.0	Plain	0.1	20000	6.280e-02	0.021598
OLS	0.0	Momentum	0.3162	9776	9.995e-05	0.018465
OLS	0.0	AdaGrad	0.1	20000	4.582e-02	0.020237
OLS	0.0	RMSProp	0.003162	20000	3.854e-02	0.018998
OLS	0.0	Adam	1	8675	9.959e-05	0.018466
Ridge	0.1	Plain	0.1	118	9.890e-05	0.057881
Ridge	0.1	Momentum	0.3162	76	4.158e-05	0.057907
Ridge	0.1	AdaGrad	0.3162	109	9.325e-05	0.057739
Ridge	0.1	RMSProp	0.03162	88	8.968e-05	0.057736
Ridge	0.1	Adam	0.1	81	3.614e-05	0.057772
Lasso	0.001	Plain	0.1	20000	2.005e-03	0.023948
Lasso	0.001	Momentum	0.3162	1781	9.652e-05	0.022898
Lasso	0.001	AdaGrad	0.1	20000	1.102e-03	0.023497
Lasso	0.001	RMSProp	0.001	20000	8.676e-04	0.023114
Lasso	0.001	Adam	0.1	1850	8.831e-05	0.022916
Lasso	0.01	Plain	0.1	4164	8.363e-05	0.039484
Lasso	0.01	Momentum	0.03162	745	7.974e-05	0.039498
Lasso	0.01	AdaGrad	0.1	7816	9.242e-05	0.039500
Lasso	0.01	RMSProp	0.0003162	2617	8.701e-05	0.039673
Lasso	0.01	Adam	0.001	3560	8.628e-05	0.039754
Lasso	0.1	Plain	0.0003162	16589	7.433e-05	0.075573
Lasso	0.1	Momentum	0.0001	4367	9.661e-05	0.075585
Lasso	0.1	AdaGrad	0.003162	16074	8.481e-05	0.075570
Lasso	0.1	RMSProp	0.0001	20000	1.603e-03	0.075596
Lasso	0.1	Adam	0.0001	3910	9.808e-05	0.075580

TABLE XX: Degree-15 selections by lowest final training cost gap. Test MSE does not select the rate.

Model	λ	Method	Rate	Updates	Gap	Test MSE
OLS	0.0	Plain	0.1	20000	7.410e-02	0.030588
OLS	0.0	Momentum	0.1	20000	4.236e-02	0.028119
OLS	0.0	AdaGrad	0.1	20000	7.969e-02	0.028303
OLS	0.0	RMSProp	0.001	20000	7.346e-02	0.030865
OLS	0.0	Adam	1	20000	3.761e-02	0.026044
Ridge	0.1	Plain	0.1	131	9.580e-05	0.058457
Ridge	0.1	Momentum	0.03162	85	6.076e-05	0.058390
Ridge	0.1	AdaGrad	0.03162	244	9.795e-05	0.058481
Ridge	0.1	RMSProp	0.003162	180	6.650e-05	0.058325
Ridge	0.1	Adam	0.1	99	3.119e-05	0.058142
Lasso	0.001	Plain	0.1	20000	7.758e-04	0.022514
Lasso	0.001	Momentum	0.1	3033	9.941e-05	0.022504
Lasso	0.001	AdaGrad	0.1	20000	2.608e-03	0.022933
Lasso	0.001	RMSProp	0.0003162	20000	9.752e-04	0.022556
Lasso	0.001	Adam	0.1	2139	8.920e-05	0.022460
Lasso	0.01	Plain	0.01	17443	9.839e-05	0.039712
Lasso	0.01	Momentum	0.003162	5407	9.188e-05	0.039772
Lasso	0.01	AdaGrad	0.03162	17074	9.150e-05	0.039436
Lasso	0.01	RMSProp	0.0001	7356	8.805e-05	0.039646
Lasso	0.01	Adam	0.001	12457	8.788e-05	0.039429
Lasso	0.1	Plain	0.0003162	20000	9.141e-04	0.075561
Lasso	0.1	Momentum	0.0001	20000	6.909e-04	0.075571
Lasso	0.1	AdaGrad	0.001	20000	3.682e-04	0.075975
Lasso	0.1	RMSProp	0.0001	20000	2.185e-03	0.075557
Lasso	0.1	Adam	0.0001	20000	8.664e-04	0.075561

TABLE XXI: Final relative training-cost gaps for degree-8 models after 1,000 epochs, with base learning rate 0.01. Batches 5, 20 and 80 use 16,000, 4,000 and 1,000 updates, respectively. Lasso uses $\lambda = 0$, Ridge uses $\lambda = 0.1$ and Lasso uses $\lambda = 0.001$. printed zeros indicate numerical agreement with the reference.

Model	Method	Batch 5		Batch 20		Batch 80	
		Constant	Decreasing	Constant	Decreasing	Constant	Decreasing
OLS	Plain	1.051e-01	2.698e-01	1.726e-01	2.969e-01	2.361e-01	3.477e-01
	Momentum	9.668e-01	1.584e-01	9.347e-02	1.768e-01	1.226e-01	2.001e-01
	AdaGrad	2.230e-01	4.339e-01	2.305e-01	3.925e-01	2.543e-01	3.810e-01
	RMSProp	1.053e-01	1.863e-01	9.628e-02	1.622e-01	1.089e-01	1.023e-01
	Adam	1.209e-01	1.877e-01	9.792e-02	1.663e-01	9.013e-02	1.477e-01
Ridge	Plain	2.167e-03	4.663e-03	3.922e-06	1.256e-02	2.396e-04	3.915e-02
	Momentum	5.989e+00	4.994e-07	1.378e-03	3.385e-07	0.000e+00	2.321e-07
	AdaGrad	3.126e-05	1.097e-01	7.018e-05	6.832e-02	7.893e-04	6.209e-02
	RMSProp	4.925e-02	1.061e-06	7.072e-03	1.286e-06	2.041e-02	5.071e-05
	Adam	1.840e-01	3.348e-07	1.500e-02	5.805e-07	1.102e-08	0.000e+00
Lasso	Plain	2.421e-02	1.807e-01	7.768e-02	2.049e-01	1.382e-01	2.614e-01
	Momentum	1.278e+00	6.545e-02	5.486e-03	8.129e-02	3.356e-02	1.025e-01
	AdaGrad	1.214e-01	3.668e-01	1.324e-01	3.113e-01	1.605e-01	3.006e-01
	RMSProp	5.141e-02	8.231e-02	8.195e-03	6.863e-02	2.251e-02	1.426e-02
	Adam	5.655e-02	9.048e-02	1.323e-02	7.208e-02	3.577e-03	5.535e-02

TABLE XXII: OLS: all saved degree-8 runs, base rate 0.01, 1,000 epochs. Batch 80 is full-batch descent.

Method	Schedule	Batch	Final gap	Updates	Seconds
Plain	constant	5	1.051e-01	16000	0.336
Plain	decreasing	5	2.698e-01	16000	0.291
Plain	constant	20	1.726e-01	4000	0.079
Plain	decreasing	20	2.969e-01	4000	0.086
Plain	constant	80	2.361e-01	1000	0.022
Plain	decreasing	80	3.477e-01	1000	0.023
Momentum	constant	5	9.668e-01	16000	0.310
Momentum	decreasing	5	1.584e-01	16000	0.313
Momentum	constant	20	9.347e-02	4000	0.089
Momentum	decreasing	20	1.768e-01	4000	0.090
Momentum	constant	80	1.226e-01	1000	0.024
Momentum	decreasing	80	2.001e-01	1000	0.024
AdaGrad	constant	5	2.230e-01	16000	0.318
AdaGrad	decreasing	5	4.339e-01	16000	0.319
AdaGrad	constant	20	2.305e-01	4000	0.088
AdaGrad	decreasing	20	3.925e-01	4000	0.102
AdaGrad	constant	80	2.543e-01	1000	0.026
AdaGrad	decreasing	80	3.810e-01	1000	0.026
RMSProp	constant	5	1.053e-01	16000	0.352
RMSProp	decreasing	5	1.863e-01	16000	0.362
RMSProp	constant	20	9.628e-02	4000	0.110
RMSProp	decreasing	20	1.622e-01	4000	0.105
RMSProp	constant	80	1.089e-01	1000	0.027
RMSProp	decreasing	80	1.023e-01	1000	0.027
Adam	constant	5	1.209e-01	16000	0.419
Adam	decreasing	5	1.877e-01	16000	0.433
Adam	constant	20	9.792e-02	4000	0.113
Adam	decreasing	20	1.663e-01	4000	0.112
Adam	constant	80	9.013e-02	1000	0.028
Adam	decreasing	80	1.477e-01	1000	0.029

TABLE XXIII: Ridge: all saved degree-8 runs, base rate 0.01, 1,000 epochs. Batch 80 is full-batch descent.

Method	Schedule	Batch	Final gap	Updates	Seconds
Plain	constant	5	4.936e-05	16000	0.265
Plain	decreasing	5	4.262e-03	16000	0.269
Plain	constant	20	3.922e-06	4000	0.075
Plain	decreasing	20	1.256e-02	4000	0.077
Plain	constant	80	2.396e-04	1000	0.020
Plain	decreasing	80	3.915e-02	1000	0.021
Momentum	constant	5	5.989e+00	16000	0.282
Momentum	decreasing	5	4.994e-07	16000	0.294
Momentum	constant	20	1.378e-03	4000	0.079
Momentum	decreasing	20	3.385e-07	4000	0.081
Momentum	constant	80	0.000e+00	1000	0.021
Momentum	decreasing	80	2.321e-07	1000	0.022
AdaGrad	constant	5	3.126e-05	16000	0.313
AdaGrad	decreasing	5	1.097e-01	16000	0.401
AdaGrad	constant	20	7.018e-05	4000	0.088
AdaGrad	decreasing	20	6.832e-02	4000	0.090
AdaGrad	constant	80	7.893e-04	1000	0.023
AdaGrad	decreasing	80	6.209e-02	1000	0.024
RMSProp	constant	5	4.925e-02	16000	0.406
RMSProp	decreasing	5	1.061e-06	16000	0.408
RMSProp	constant	20	7.072e-03	4000	0.105
RMSProp	decreasing	20	1.286e-06	4000	0.105
RMSProp	constant	80	2.041e-02	1000	0.027
RMSProp	decreasing	80	5.071e-05	1000	0.027
Adam	constant	5	1.840e-01	16000	0.464
Adam	decreasing	5	3.348e-07	16000	0.450
Adam	constant	20	1.500e-02	4000	0.121
Adam	decreasing	20	5.805e-07	4000	0.119
Adam	constant	80	1.102e-08	1000	0.030
Adam	decreasing	80	0.000e+00	1000	0.030

TABLE XXIV: Lasso: all saved degree-8 runs, base rate 0.01, 1,000 epochs. Batch 80 is full-batch descent.

Method	Schedule	Batch	Final gap	Updates	Seconds
Plain	constant	5	1.675e-02	16000	0.372
Plain	decreasing	5	1.746e-01	16000	0.379
Plain	constant	20	7.768e-02	4000	0.105
Plain	decreasing	20	2.049e-01	4000	0.107
Plain	constant	80	1.382e-01	1000	0.028
Plain	decreasing	80	2.614e-01	1000	0.029
Momentum	constant	5	1.278e+00	16000	0.395
Momentum	decreasing	5	6.545e-02	16000	0.397
Momentum	constant	20	5.486e-03	4000	0.109
Momentum	decreasing	20	8.129e-02	4000	0.108
Momentum	constant	80	3.356e-02	1000	0.027
Momentum	decreasing	80	1.025e-01	1000	0.027
AdaGrad	constant	5	1.251e-01	16000	0.421
AdaGrad	decreasing	5	3.568e-01	16000	0.414
AdaGrad	constant	20	1.324e-01	4000	0.108
AdaGrad	decreasing	20	3.113e-01	4000	0.117
AdaGrad	constant	80	1.605e-01	1000	0.030
AdaGrad	decreasing	80	3.006e-01	1000	0.032
RMSProp	constant	5	2.961e-02	16000	0.433
RMSProp	decreasing	5	8.938e-02	16000	0.437
RMSProp	constant	20	8.195e-03	4000	0.114
RMSProp	decreasing	20	6.863e-02	4000	0.128
RMSProp	constant	80	2.251e-02	1000	0.033
RMSProp	decreasing	80	1.426e-02	1000	0.033
Adam	constant	5	5.655e-02	16000	0.511
Adam	decreasing	5	9.048e-02	16000	0.520
Adam	constant	20	1.323e-02	4000	0.131
Adam	decreasing	20	7.208e-02	4000	0.134
Adam	constant	80	3.577e-03	1000	0.034
Adam	decreasing	80	5.535e-02	1000	0.035

TABLE XXV: All saved CV selections, including both fold counts. Zero penalty denotes OLS; Ridge penalties retain scientific notation.

Folds	Method	Degree	Mean-cost λ	CV MSE
5	OLS	11	0	0.0211129
5	Ridge	12	1.79806e-07	0.0207671
5	Lasso	11	1e-06	0.0211418
10	OLS	11	0	0.0202373
10	Ridge	12	1.59828e-07	0.0192712
10	Lasso	13	1.43845e-05	0.0201081